Técnicas de Programação II 2026

Revisão: Variáveis, I/O, Operadores, Condicionais, Repetições

Introdução

Por que aprender uma linguagem de programação?

- Existem linguagens de propósito geral
- Existem linguagens de propósito específico
- Cada linguagem se propõe a resolver melhor alguns tipos de problema;

Ter sempre em mente o que eu quero desenvolver.

- Desenvolver jogos;
- Desenvolver sistemas embarcados;
- Trabalhar com grandes softwares comerciais;
- Trabalhar com sistemas operacionais;
- Sistemas em tempo real/alta performance;

C++: Principais Vantagens

Atende muito bem cenários complexos e que demandam informações em tempo real:

Por exemplo: sistemas embarcados, mercado financeiro, jogos, microcontroladores;

Linguagem de alto nível e realmente compilada;

Quem usa C++ hoje em dia:

Facebook;

Adobe (Photoshop/Illustrator);

Microsoft;

EA Games;

C++: Principais Desvantagens

-Existem situações onde C++ precisa de muitas linhas de código para se realizar ações simples;

Por exemplo: Sistemas Web Simples

-Possui uma curva de aprendizagem um pouco mais longa

Por que não aprender?

Aprendizado sempre é valido!

Similaridades de C e C++

- Ambas as linguagens têm uma sintaxe semelhante.
- A estrutura de código de ambas as linguagens é a mesma.
- A compilação de ambas as linguagens é semelhante.
- Eles compartilham a mesma sintaxe básica. Quase todos os operadores e palavras-chave de C também estão presentes em C++ e fazem a mesma coisa.
- C++ tem uma gramática ligeiramente estendida que C, mas a gramática básica é a mesma.
- O modelo básico de memória de ambos é muito próximo do hardware.
- As mesmas noções de pilha, heap, escopo de arquivo e variáveis estáticas estão presentes em ambas as linguagens.

Diferenças entre C e C++

C	C++
C foi desenvolvido por Dennis Ritchie entre os anos de 1969 e 1973 na AT&T Bell Labs.	C++ foi desenvolvido por Bjarne Stroustrup em 1979.
C não suporta polimorfismo, encapsulamento e herança, o que significa que C não suporta programação orientada a objetos.	C++ suporta polimorfismo , encapsulamento e herança porque é uma linguagem de programação orientada a objetos.
C é um subconjunto de C++.	C++ é um superconjunto de C.
C contém 32 palavras- chave .	C++ contém 63 palavras- chave .
Para o desenvolvimento de código, C suporta programação procedural .	C++ é conhecido como linguagem híbrida porque C++ suporta paradigmas de programação procedural e orientado a objetos .

Diferenças entre C e C++

C	C++
C foi desenvolvido por Dennis Ritchie entre os anos de 1969 e 1973 na AT&T Bell Labs.	C++ foi desenvolvido por Bjarne Stroustrup em 1979.
C não suporta polimorfismo, encapsulamento e herança, o que significa que C não suporta programação orientada a objetos.	C++ suporta polimorfismo , encapsulamento e h erança porque é uma linguagem de programação orientada a objetos.
C é um subconjunto de C++.	C++ é um superconjunto de C.
C contém 32 palavras- chave .	C++ contém 63 palavras- chave .
Para o desenvolvimento de código, C suporta programação procedural .	C++ é conhecido como linguagem híbrida porque C++ suporta paradigmas de programação procedural e orientado a objetos .
Dados e funções são separados em C porque é uma linguagem de programação procedural.	Dados e funções são encapsulados juntos na forma de um objeto em C++.

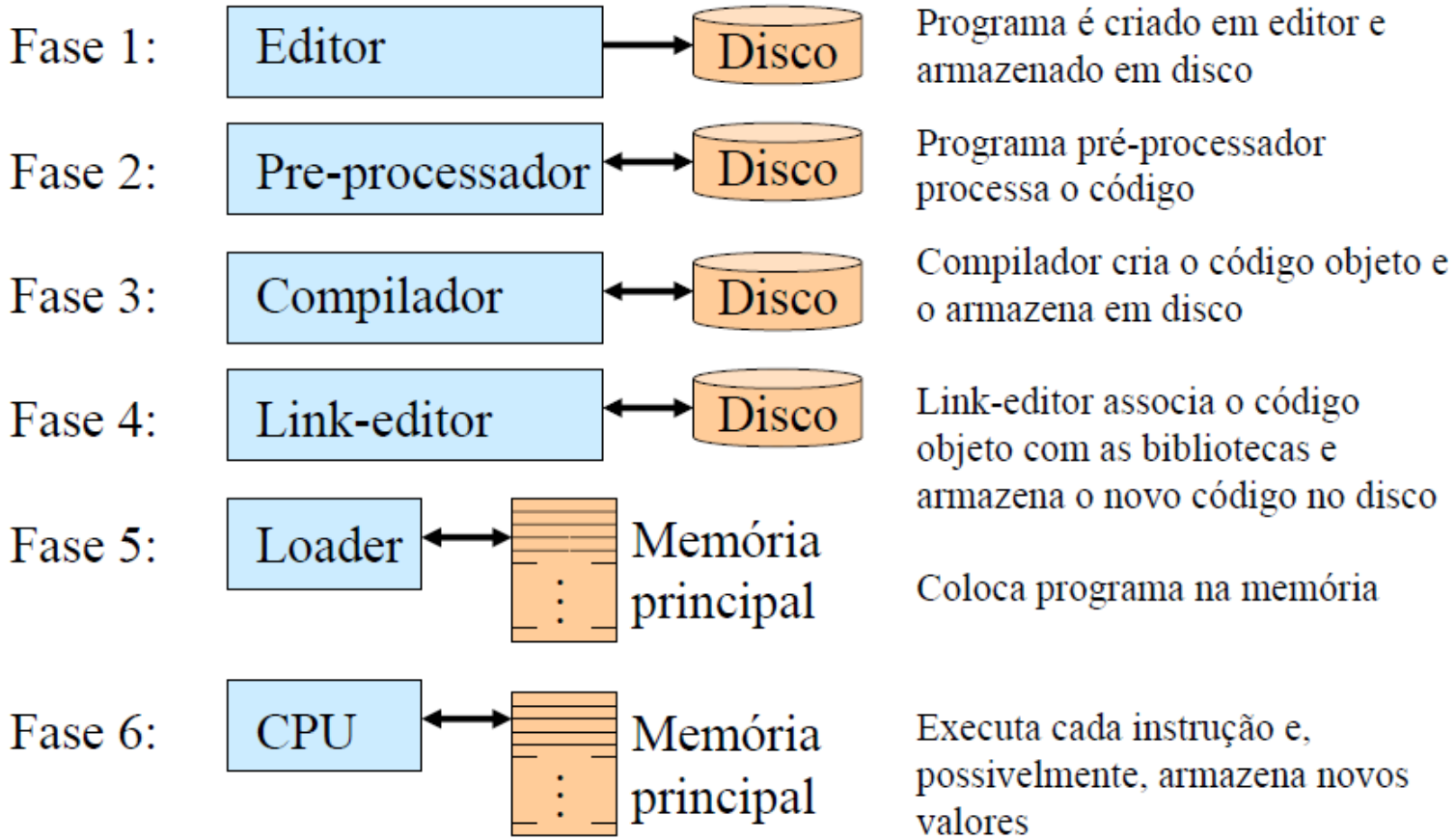
Diferenças entre C e C++

C	C++
C não suporta ocultação de informações.	Os dados são ocultados pelo Encapsulamento para garantir que as estruturas de dados e os operadores sejam usados conforme pretendido.
Tipos de dados internos são suportados em C.	Tipos de dados integrados e definidos pelo usuário são suportados em C++.
C é uma linguagem orientada a funções porque C é uma linguagem de programação procedural.	C++ é uma linguagem orientada a objetos porque é uma programação orientada a objetos.
A sobrecarga de funções e operadores não é suportada em C.	A sobrecarga de funções e operadores é suportada pelo C++.
C é uma linguagem orientada por funções.	C++ é uma linguagem orientada a objetos
Funções em C não são definidas dentro de estruturas.	As funções podem ser usadas dentro de uma estrutura em C++.
Os recursos de namespace não estão presentes dentro do C.	Namespace é usado por C++, que evita colisões de nomes.
O arquivo de cabeçalho usado por C é stdio.h .	O arquivo de cabeçalho usado pelo C++ é iostream.h .
Variáveis de referência não são suportadas por C.	As variáveis de referência são suportadas pelo C++.

Diferenças entre C e C++

C	C++
As funções virtuais e de amigos não são suportadas por C.	As funções virtuais e amigas são suportadas por C++.
C não suporta herança.	C++ suporta herança.
Em vez de se concentrar em dados, C se concentra em método ou processo.	C++ se concentra em dados em vez de se concentrar em método ou procedimento.
C fornece funções malloc() e calloc() para alocação dinâmica de memória e free() para desalocação de memória.	C++ fornece operador new para alocação de memória e operador delete para desalocação de memória.
O suporte direto para tratamento de exceção não é suportado pelo C.	O tratamento de exceção é suportado pelo C++.
As funções scanf() e printf() são usadas para entrada/saída em C.	cin e cout são usados para entrada/saída em C++ .
Estruturas C não possuem modificadores de acesso.	Estruturas C++ possuem modificadores de acesso

Criação de Programas em C++



Erros de Programação

- **Muitas vezes, os programadores cometem erros ao escrever seus programas**

- **Em geral, um programa pode apresentar 4 categorias de erros**
  - Erros sintáticos ou de compilação
  - Erros de composição
  - Erros de execução
  - Erros semânticos

Erros de Programação

- **Erros sintáticos ou de compilação**

Código não obedece às regras de sintaxe da linguagem – não compila

»Sintaxe de uma linguagem é a sua “gramática”

Erros são indicados através de mensagens de erros resumidas na tela do computador

»Mensagens de erros mais detalhadas podem ser encontradas em manuais da linguagem

»Feitas as correções necessárias, a compilação pode ser reinicializada

Erros de Programação

- **Erros de composição**

Ocorrem na fase de link-edição

Geralmente estão relacionados a erros na chamada de funções em outras bibliotecas

»Exemplo: chamar a função de saída de rand() sem importar a bibliotecas usadas

Erros são indicados através de mensagens de erros resumidas na tela do computador

Erros de Programação

- **Erros de execução**

Código obedece às regras de linguagem

»programa é compilado

Mas ao ser executado, código executa operações não permitidas

»Exemplos

- dividir por zero
- raiz quadrada de número negativo

São mais difíceis de descobrir e interpretar

Erros de Programação

- **Erros semânticos**

Semântica = significado

Erros no projeto lógico do programa

»Forma utilizada para resolver o problema

Provoca erros indesejáveis no programa

»Mensagens indicativas dos erros não são apresentadas

»Sua correção exige mudanças na concepção do programa

São os erros de detecção mais difícil

Variáveis em C++

Tipos fundamentais:

bool - Boolean value, either true or false

char - caracter

short - short integer

int - integer

long - long integer

float - single-precision floating-point number

double - double-precision floating-point number

void - indicate the absence of any type

Variáveis em C++

Definição de Variáveis:

- **Toda variável utilizada no programa deve ser declarada**
- **Sintaxe para a declaração de variáveis em C++**

tipo lista_de_nomes;

- **Em C++, o valor inicial de uma variável é indefinido**
  - Exceção: *int result = 0;*
- **Valor inicial de uma declaração é chamado de inicializador**

Variáveis em C++

- **Convenções para nomes**

Identificadores: nomes usados para variáveis, constantes, tipos e funções

Regras

»Nome deve iniciar com letra ou *underscore* (_)

»Caracteres de um nome devem ser letras, números ou *underscores*

»Palavras chaves não podem ser utilizadas como nomes (https://en.cppreference.com/w/cpp/keyword)

Variáveis em C++

- **Convenções para nomes**

Letras maiúsculas e minúsculas são consideradas **diferentes**

Estilo de programação pode ser melhorado adotando convenções

»Exemplo:

- Nomes de variáveis e tipos de dados começam com letras minúsculas
- Nomes de funções e constantes começam com letras MAIÚSCULAS

I/O em C++

Biblioteca padrão de entrada e saída

<iostream.h>

Inclui funções para a entrada (leitura) e saída (escrita) de dados;

```
cout << a;
cin >> b;
```

É possível manipular I/O com **modificadores**.

I/O em C++

Modificadores:

dec – formata para decimal

hex – formata para hexadecimal

oct – formata para octadecimal

endl (\n) – fim de linha

I/O em C++

```
int main()
{
   cout << “Número 42 em octal:   " << oct << 42 << '\n'
          << “Número 42 em decimal: " << dec << 42 << '\n'
           << “Número 42 em hex:     " << hex << 42 << '\n';
}
/*Saída
Número 42 em octal: 52
Número 42 em decimal: 42
Número 42 em hex: 2a
*/
```

Operadores

Standard algebraic equality or relational operator	C++ equality or relational operator	Sample C++ condition	Meaning of C++ condition
Relational operators			
>	>	x > y	x is greater than y
<	<	x < y	x is less than y
≥	>=	x >= y	x is greater than or equal to y
≤	<=	x <= y	x is less than or equal to y
Equality operators			
=	==	x == y	x is equal to y
≠	!=	x != y	x is not equal to y

Operadores

Operators				Associativity	Type
*	/	%		left to right	multiplicative
+	-			left to right	additive
<<	>>			left to right	stream insertion/extraction
<	<=	>	>=	left to right	relational
==	!=			left to right	equality
=				right to left	assignment

Condicionais

- **São estruturas de controle condicionais**

Muitas vezes, a avaliação de uma condição define a execução ou não de um comando (execução condicional)

A maneira mais fácil de fazer isso é através do comando *If*

```
if (condicao)
        comando;
```

```
if (condicao)
        comando1 ;
else
        comando2;
```

Comandos podem ser simples ou compostos (bloco)

Condicionais

- **Operadores lógicos podem ser utilizados na equação de condição**

–Exemplos:

```
...
if ( (a>5) && (b==-1) ) {
    ...
```

```
...
if ( (a>1) || !(b==2) ) {
    ...
```

Repetições

- **Permitem a execução de partes de um programa mais de uma vez**

- **São comandos iterativos da linguagem C++**

–Comando *while*

–Comando *do while*

–Comando *for*

Repetições (while)

- **Estrutura de repetição mais simples**
- **Executa um comando repetidamente até uma condição se tornar falsa**

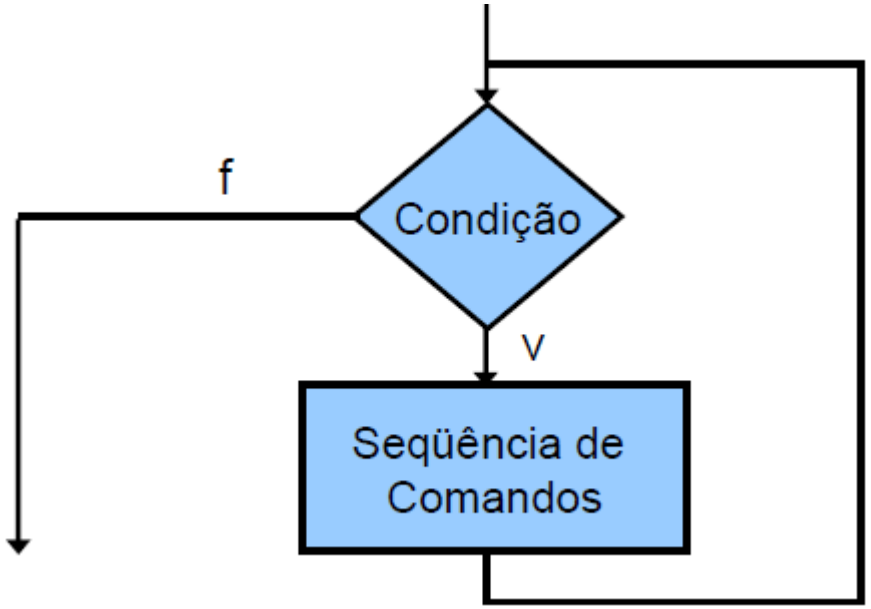
```
while (expressão condicional)
        comando;
```

Comandos podem ser simples ou compostos (bloco)

Repetições (while)

- **Teste condicional é executado antes de cada ciclo do *loop***

–Se o primeiro teste resultar no valor 0 (Falso), o corpo do loop não é executado



Repetições (while)

- **Utilizado onde existe uma condição de teste que possa ser aplicada antes da execução do corpo**
- **Vários problemas de programação não se encaixam na estrutura do comando *while***
  - As vezes o teste seria mais natural em algum lugar no meio do loop
- Ex.: leitura de dados do usuário até o recebimento de um valor especial (sentinela)

Repetições (while)

- **Utilizado onde existe uma condição de teste que possa ser aplicada antes da execução do corpo**
- **Vários problemas de programação não se encaixam na estrutura do comando *while***
  - As vezes o teste seria mais natural em algum lugar no meio do loop
- Ex.: leitura de dados do usuário até o recebimento de um valor especial (sentinela)
- ***Loop* baseado em sentinela**
- Ler um valor
- Se o valor é igual ao sentinela, sair do *loop*
- Senão, executar o processamento requerido por este valor

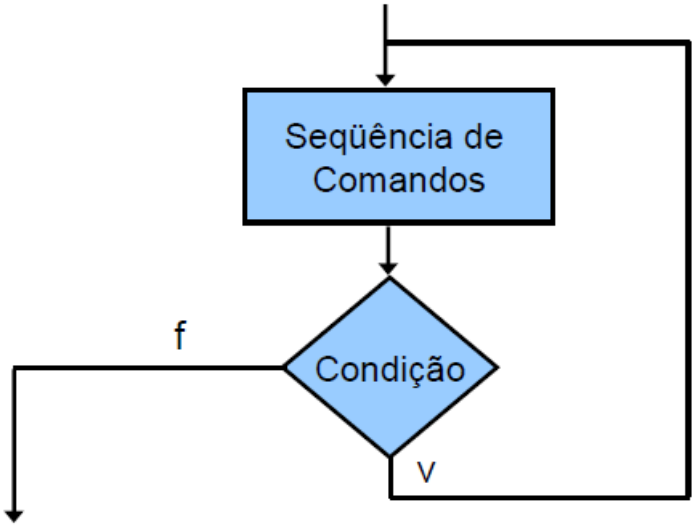
Repetições (while)

- **Executa um comando repetidamente até uma condição se tornar falsa**

do {

comandos;

} while (expressao condicional);



- **Semelhante ao comando *while***

–Exceto que a expressão é avaliada no final

Repetições (for)

- **O comando *for* é um comando de repetição determinado pelas expressões:**

–inicial

–teste

–passo

```
for (inicial; teste; passo) {
        comandos;
}
```

Repetições (for)

- **Inicial**

–Expressão que indica como o *loop* do comando *for* deve ser inicializado

–Executado uma única vez no início do *loop*

–Define o valor inicial da variável de indexação (contador)

- Ex.: for (i = 0; .... for (i = -7; ...

Repetições (for)

- **Teste**

–Expressão que indica quando o *loop* do comando *for* deve parar

–Funciona como a condição de teste do *while*

–Expressão é avaliada no topo de cada interação do *loop*

- Compara valor do contador com um valor final
  - –Quando o resultado da avaliação é *FALSO*, o *loop* termina
  - –Enquanto teste é VERDADEIRO, o *loop* continua
- Ex.: for (i = 0; (i < n); i++){

Repetições (for)

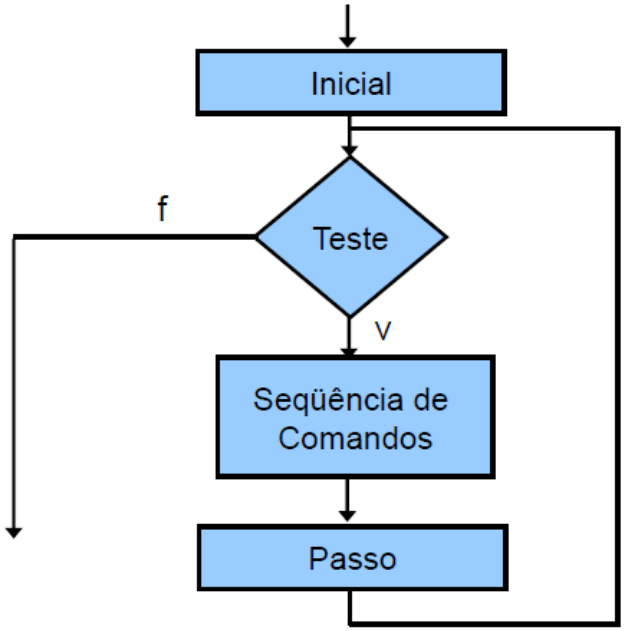
- **Passo**

–Expressão chamada a cada interação do *loop* para atualizar o valor do contador

–Indica quanto muda o valor da variável de indexação de um ciclo para outro

–Especificações de passo mais comuns

- *index++, index--*



Repetições (for)

- **Expressões inicial, teste e passo são opcionais**

–Os separadores (;) devem aparecer

–Falta de valor inicial -> não é feita nenhuma inicialização do indexador

–Falta de condição de teste -> ela é assumida como sempre VERDADEIRA

–Falta de passo -> indexador não é alterado entre ciclos do loop

- Mas pode ser alterado dentro do ciclo